

Worksheet 5 — Cross-Site Scripting & Client-Side Risks (3 hrs)

Course: Software Security (KOSEN69) · **Week 5 Aligned:** OWASP 2025 A05 Injection · **CWE-79** (XSS), **CWE-352** (CSRF), **CWE-1004** (cookie without HttpOnly) **Signature game:** 🚩 **XSS Golf** — fire `alert(1)` in the fewest characters possible. Lower payload length = lower score = better. Par for reflected is the `<img>` vector; can you go under par?

⚠️ **Ethics note:** Use only the provided `vulnerable_app.py` sandbox and your own Juice Shop container. Stealing real users' cookies or sessions is illegal. All "session theft" steps here target the sandbox cookie `session=abc123` only.

Part 1 — Student Information

Name	Student ID	Date	Group
Sutapat Chucham	6531503085	15 Sep 2026	

Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **reflected**, **stored**, and **DOM-based** XSS by *where* the untrusted data is injected and *when* it executes. Which two does our `vulnerable_app.py` implement, and at which routes?

Ans: Reflected, stored, and DOM-based XSS differ in payload placement and execution. Reflected XSS bounces the request payload directly into the response once, so the attacker needs the victim to click a crafted link; stored XSS saves the payload in server-side state, so it re-fires for every visitor without link-clicking; and DOM-based XSS never touches the server and reads untrusted data from `location.write` hash to a sink like `innerHTML` to execute totally in the browser. Reflected XSS at `/hello` (name parameter is echoed unescaped) and stored XSS at `/comments` (submitted comments are saved and presented raw to every visitor) are implemented in `vulnerable_app.py`.

2. How does **contextual output encoding** (`markupsafe.escape`) stop `<script>` from executing? Why is HTML-context encoding different from JavaScript- or URL-context encoding?

Ans: When you use `escape()`, special characters in HTML (`<`, `>`, `&`, `"`, `'`) are turned into HTML entities. This means that stays and the parser never starts a real element. The data stays data and never turns into code. This does not change anything about a JS engine or URL parser, though; it only changes data for the HTML parser. The same string could still break out of a JS string literal if it ended up in a block instead. This is because HTML entities do not work there. This is the reason why HTML, JS, URL, and CSS all need their own encoders that work with them.

3. Explain how a strict **Content-Security-Policy** (`script-src 'self'`) defeats an *injected* inline script even when encoding is missing.

Ans: This is a list of script sources that the browser can run. It is sent through the Content-Security-Policy header and is enforced by the browser. By default, `script-src "self"` doesn't include inline tags or inline

handlers like `onerror`, unless `"unsafe-inline"` or a nonce is added. This means that even if an unencoded payload is added to the DOM, the browser will not run the inline code. This stops the attack in two separate steps, one after the other.

4. What do the cookie flags **HttpOnly**, **SameSite**, and **Secure** each protect against? Map each to a concrete attack (cookie theft via XSS, CSRF, network sniffing).

Ans: `HttpOnly` blocks document-based cookie reading by `JavaScript.cookie`, so an XSS payload in the victim's browser can't steal the session cookie (CWE-1004). `SameSite (Lax/Strict)` prevents cross-site requests from attaching cookies, so a forged CSRF form or image tag from another site arrives at the server without the victim's session cookie. `Secure` always sends the cookie via HTTPS, so network sniffers can't intercept a plaintext copy.

5. Why does **CSRF** (CWE-352) work even without any script injection, and how does `SameSite=Strict` plus the same-origin policy blunt it?

Ans: CSRF doesn't require script injection since browsers immediately attach a user's stored cookies to any request submitted to that cookie's domain, independent of the page or origin. For logged-in victims, an auto-submitting form or element pointing to the vulnerable endpoint can trigger an authenticated request. `SameSite=Strict` explicitly refuses to attach the cookie to any request from a different site, therefore the falsified request arrives at the server without a session cookie and no authentication. The same-origin policy prevents the attacker's script from reading back cross-origin answers, blocking multi-step CSRF that requires to steal a token, but it does not prevent the initial fraudulent request.

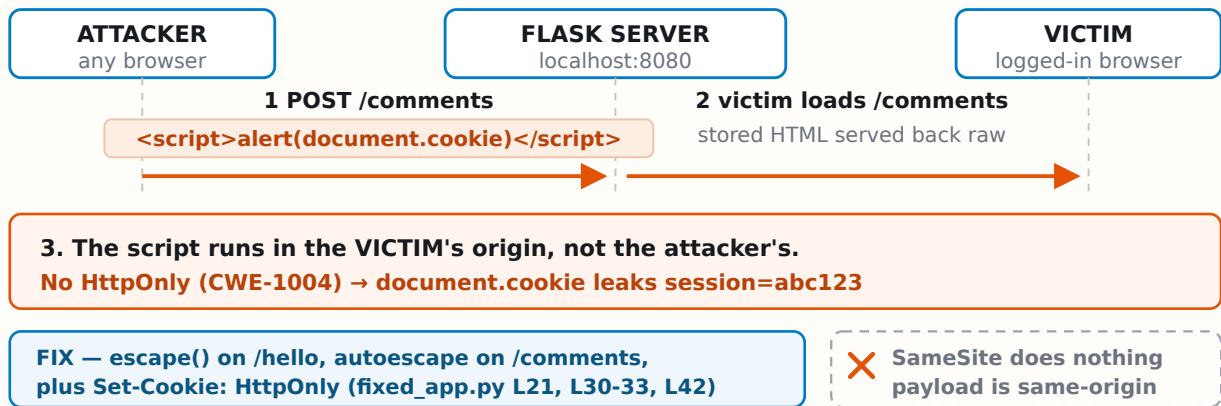
Part 3 — Hands-on Lab (150 min)

Same browser, same cookie, opposite directions

1. STORED XSS (cross-site scripting) · CWE-79

attacker → server → victim

Written once by the attacker, run in every visitor's browser.



2. CSRF (cross-site request forgery) · CWE-352

victim → attacker page → server

No script injected. The victim's own browser sends the request.



- fixed_app.py adds encoding, CSP and HttpOnly — the XSS dies.
- The CSRF still works: /comments checks no token on POST.

Learning goals: land reflected + stored XSS, abuse a JS-readable cookie, build a CSRF PoC against the comment board, then prove `fixed_app.py` blocks all of it.

Prerequisites: Docker + Docker Compose, a browser with DevTools, a text editor. Working dir: `labs/week05-xss-client-side/`.

Environment setup

```

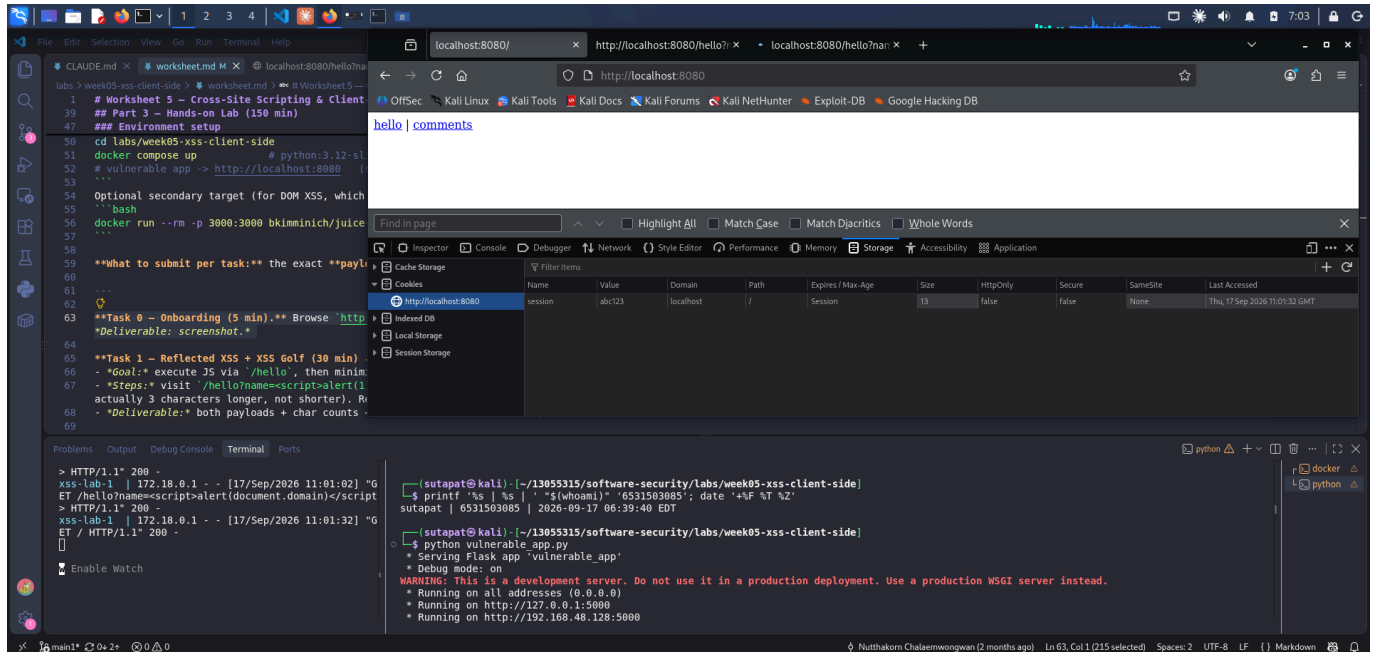
cd labs/week05-xss-client-side
docker compose up          # python:3.12-slim + flask, runs
vulnerable_app.py
# vulnerable app -> http://localhost:8080   (service name: xss-lab, port
8080)
  
```

Optional secondary target (for DOM XSS, which our app does not expose):

```
docker run --rm -p 3000:3000 bkimminich/juice-shop # ->
http://localhost:3000
```

What to submit per task: the exact **payload**, a **screenshot** of the alert/effect, and a **2–3 sentence mitigation**.

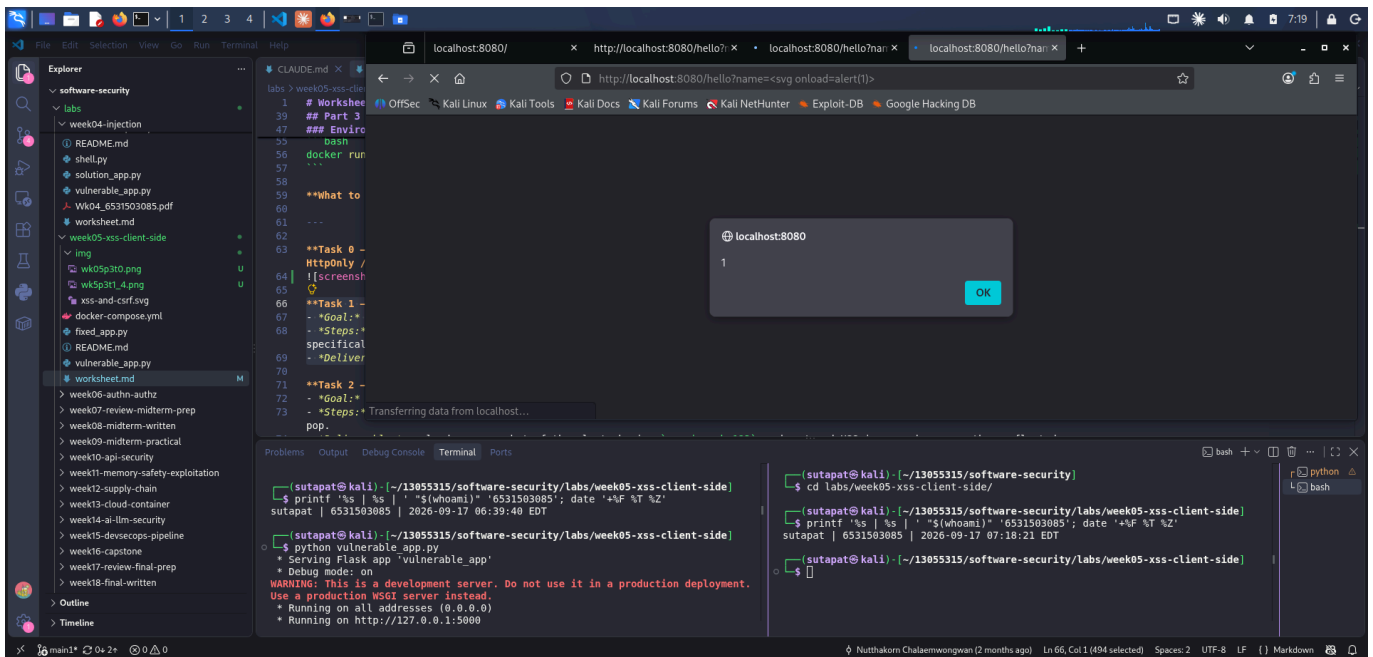
Task 0 — Onboarding (5 min). Browse `http://localhost:8080/`. Open DevTools → Application → Cookies and confirm `session=abc123` is set with **no HttpOnly / SameSite**. Screenshot it. *Deliverable: screenshot.*



Task 1 — Reflected XSS + XSS Golf (30 min) 🚩.

- Goal:** execute JS via `/hello`, then minimize the payload.
- Steps:** visit `/hello?name=<script>alert(1)</script>`, then the alternate `/hello?name=<img src=x onerror=alert(1)>` (useful when `<script>` tags specifically are filtered — note it's actually 3 characters longer, not shorter). Record each payload's character count for your golf score.
- Deliverable:** both payloads + char counts + screenshot of `alert(1)` + your lowest score.

Payload	Char count
<code><script>alert(1)</script></code>	25
<code></code>	28
<code><svg onload=alert(1)></code>	21 (lower score)

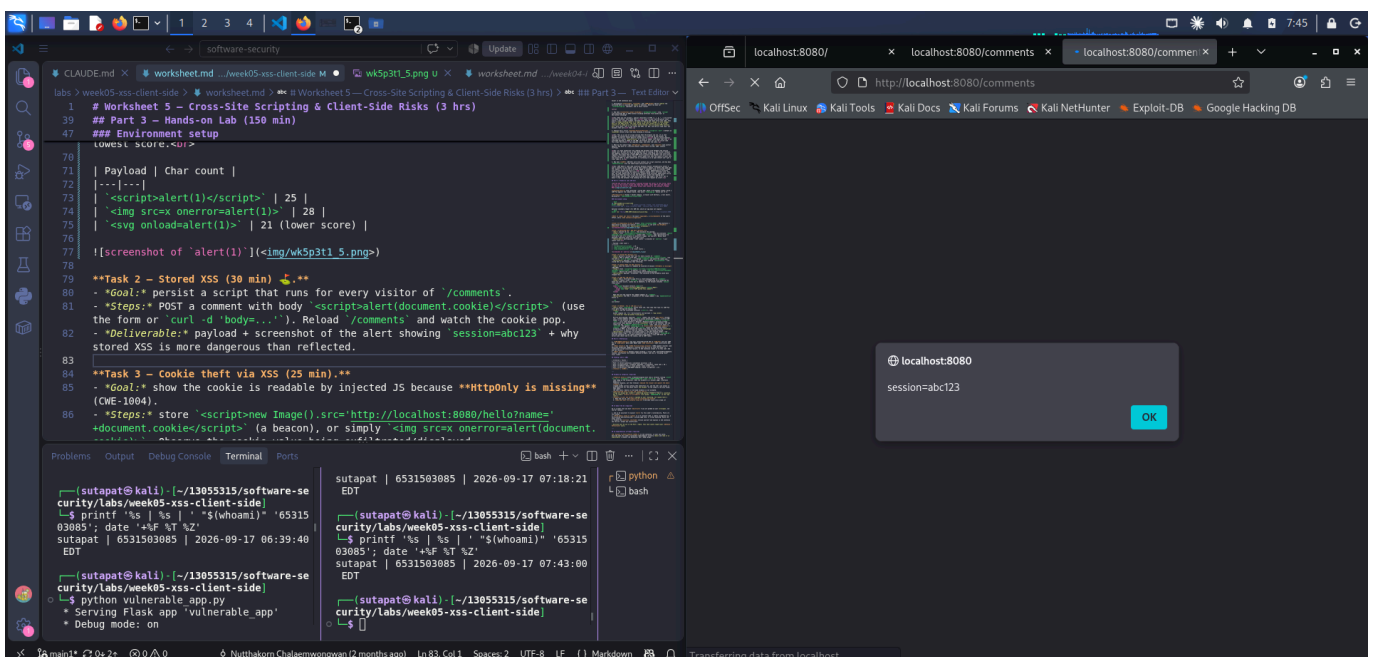


Task 2 — Stored XSS (30 min) 🚩.

- **Goal:** persist a script that runs for every visitor of `/comments`.
- **Steps:** POST a comment with body `<script>alert(document.cookie)</script>` (use the form or `curl -d 'body=...'`). Reload `/comments` and watch the cookie pop.
- **Deliverable:** payload + screenshot of the alert showing `session=abc123` + why stored XSS is more dangerous than reflected.

Payload: `<script>alert(document.cookie)</script>`

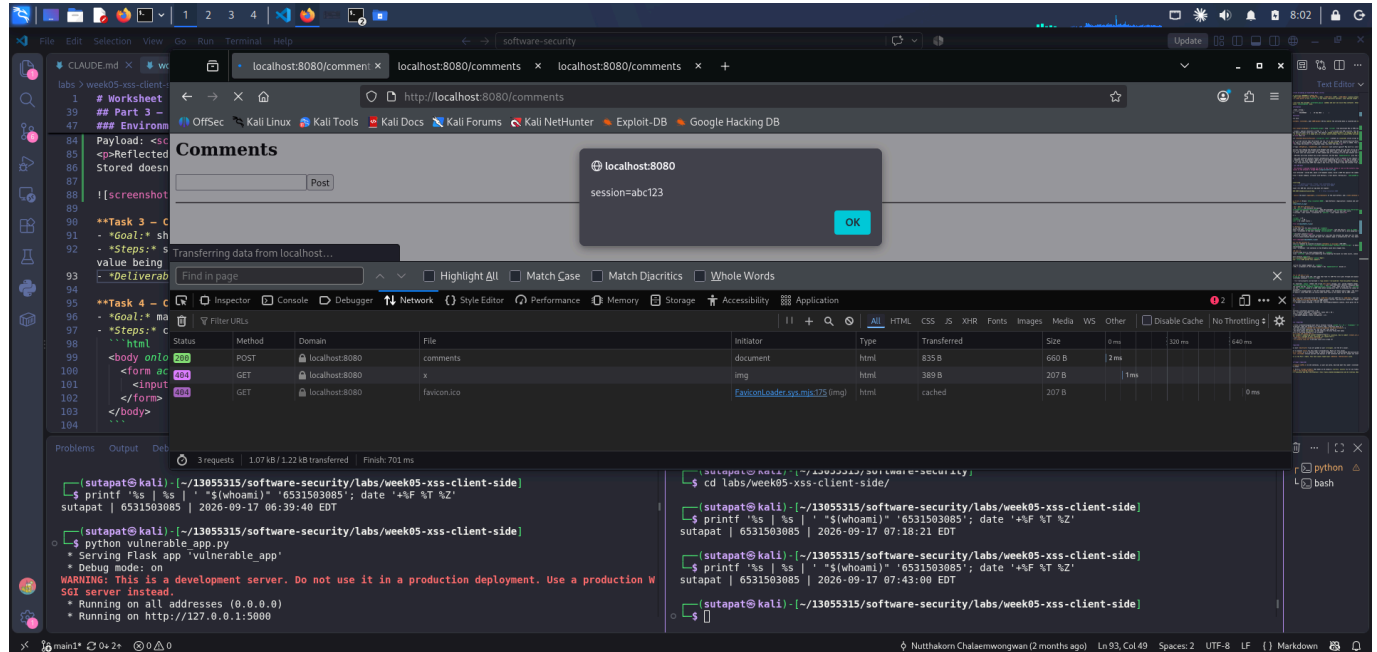
Reflected targets people by "tricking" them into clicking on a link that the attacker has made just for them. Only people who click that link are harmed. Stored doesn't have to trick anyone because anyone who opens the `/comments` page is automatically hit. This means that everyone is a victim, not just the people who are being targeted.



Task 3 — Cookie theft via XSS (25 min).

- **Goal:** show the cookie is readable by injected JS because **HttpOnly** is missing (CWE-1004).
- **Steps:** store `<script>new Image().src='http://localhost:8080/hello?name='+document.cookie</script>` (a beacon), or simply `<img src=x onerror=alert(document.cookie)>`. Observe the cookie value being exfiltrated/displayed.
- **Deliverable:** payload + screenshot + 2–3 sentences on how HttpOnly would have stopped this.

payload: `<script>new Image().src='http://localhost:8080/hello?name='+document.cookie</script>`



JavaScript cannot access document cookies with HttpOnly. Cookie, the browser hides it from site scripts. If our session cookie was HttpOnly, our injected payload would document.cookie would have returned an empty value or omitted the session cookie, preventing exfiltration despite XSS execution. This is because HttpOnly just stops cookie theft, not the underlying XSS vulnerability, hence proper output encoding is still the best solution as the script might still cause harm while acting as the victim.

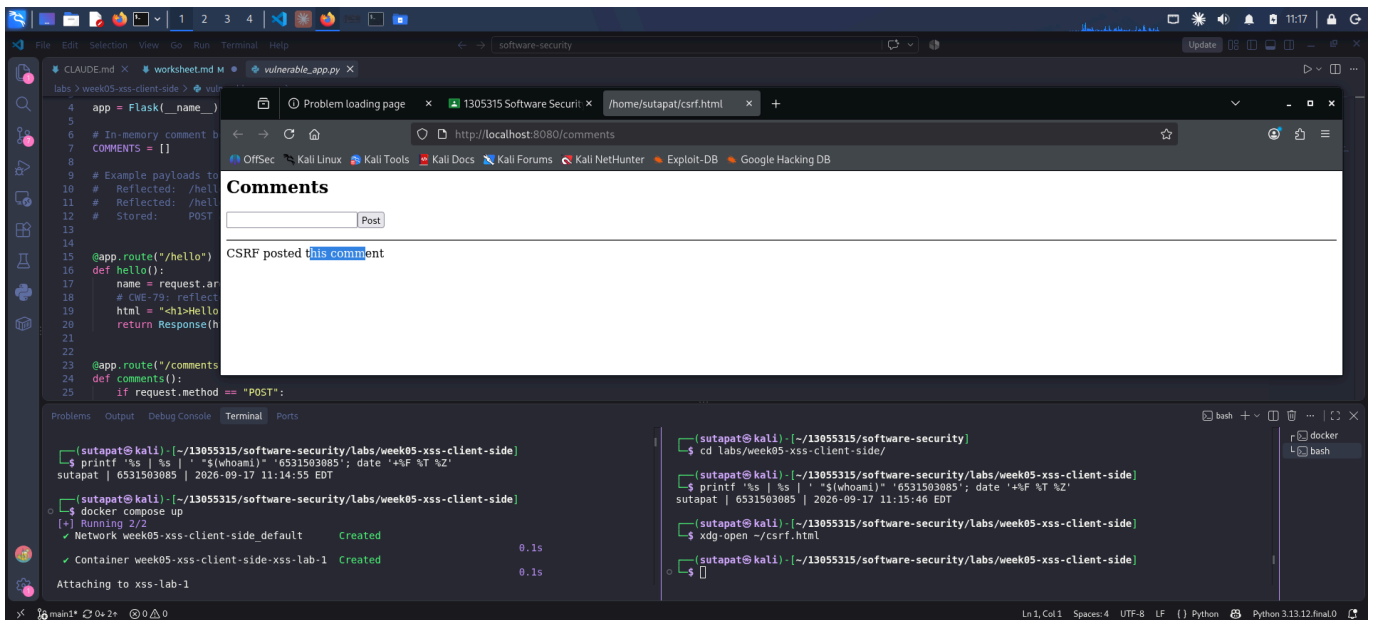
Task 4 — CSRF PoC (30 min).

- **Goal:** make a third-party page force a state-changing POST to `/comments`.
- **Steps:** create a local `csrf.html` with an auto-submitting form targeting the board (no token exists, cookie has no SameSite, so the browser attaches `session` cross-site):

```
<body onload="document.forms[0].submit()">
  <form action="http://localhost:8080/comments" method="POST">
    <input name="body" value="CSRF posted this comment">
  </form>
</body>
```

Open the file and confirm the comment appears on `/comments`.

- **Deliverable:** the HTML + screenshot of the forged comment + why `SameSite=Strict` blocks it.



When you set SameSite=Strict, the browser will not attach a cookie unless the request comes from the same site as the cookie's domain. The browser doesn't give the fake POST the session cookie because csrf.html comes from a file:// page, which is not the same as localhost:8080. This means that the request arrives without any name attached. This wouldn't stop the comment from being saved, though, because /comments never checks for a session cookie or CSRF token to begin with. It only stops the fake request from being linked to a real user, which Task 5 confirms directly.

xss-context

Task 5 — Defend / fix it (30 min) 🛡️.

- **Goal:** prove `fixed_app.py` blocks Tasks 1–3, then show that Task 4's CSRF PoC still gets through and explain why.
- **Steps:** stop the vulnerable container (`Ctrl-C`), then:

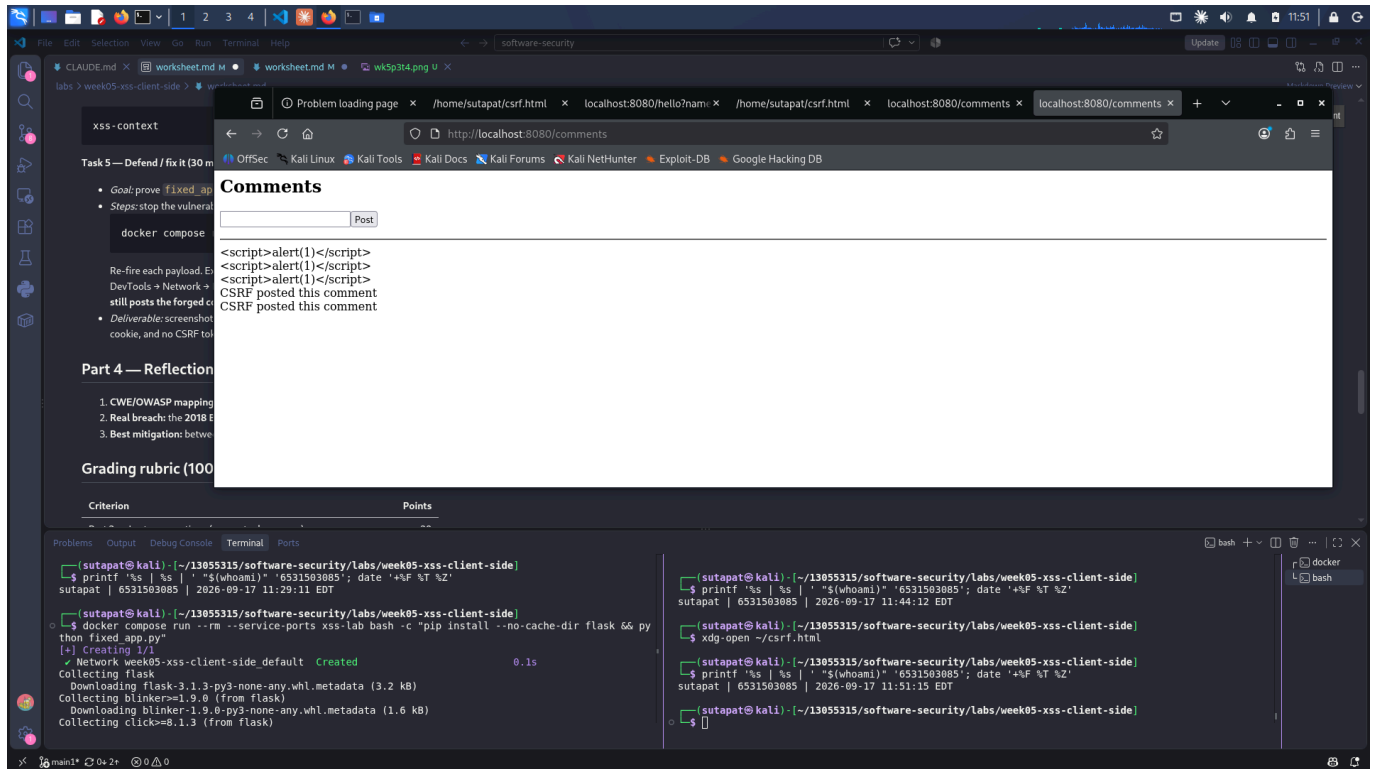
```
docker compose run --rm --service-ports xss-lab bash -c "pip install -no-cache-dir flask && python fixed_app.py"
```

Re-fire each payload. Expected: `/hello` renders the script as text (escape, L21), stored comments render literally (Jinja autoescape, L30–33), a strict CSP header is now present as defense-in-depth (`Content-Security-Policy: script-src 'self'`, L12 — check DevTools → Network → Response Headers; escaping already neutralizes these payloads, so no CSP violation fires in the console), and the cookie now has `HttpOnly; SameSite=Strict; Secure` (L42). Then re-run Task 4's `csrf.html` PoC against `fixed_app.py`: it still posts the forged comment — `/comments` (L25–28) never checks the `session` cookie or a CSRF token before accepting a POST, so hardening the cookie only stops the browser from attaching it cross-site; it doesn't stop the request itself from being processed.

- **Deliverable:** screenshots of escaped output + the CSP response header + the hardened cookie flags + the still-successful Task 4 forgery against `fixed_app.py`, with 2–3 sentences on why cookie

hardening alone doesn't close CSRF here (no server-side check tied to the cookie, and no CSRF token).

The /comments route still handles the fake POST even when SameSite is set to strict. This is because the server-side code never checks the session cookie or a CSRF token before saving the comment. SameSite only manages whether the browser adds the cookie; this is a client-side behaviour. The server-side decision about authorisation takes place, and there is none here. To fully fix the problem, both cookie hardening and a server-side CSRF token check are needed. The server-side CSRF token check acts as the main defence.



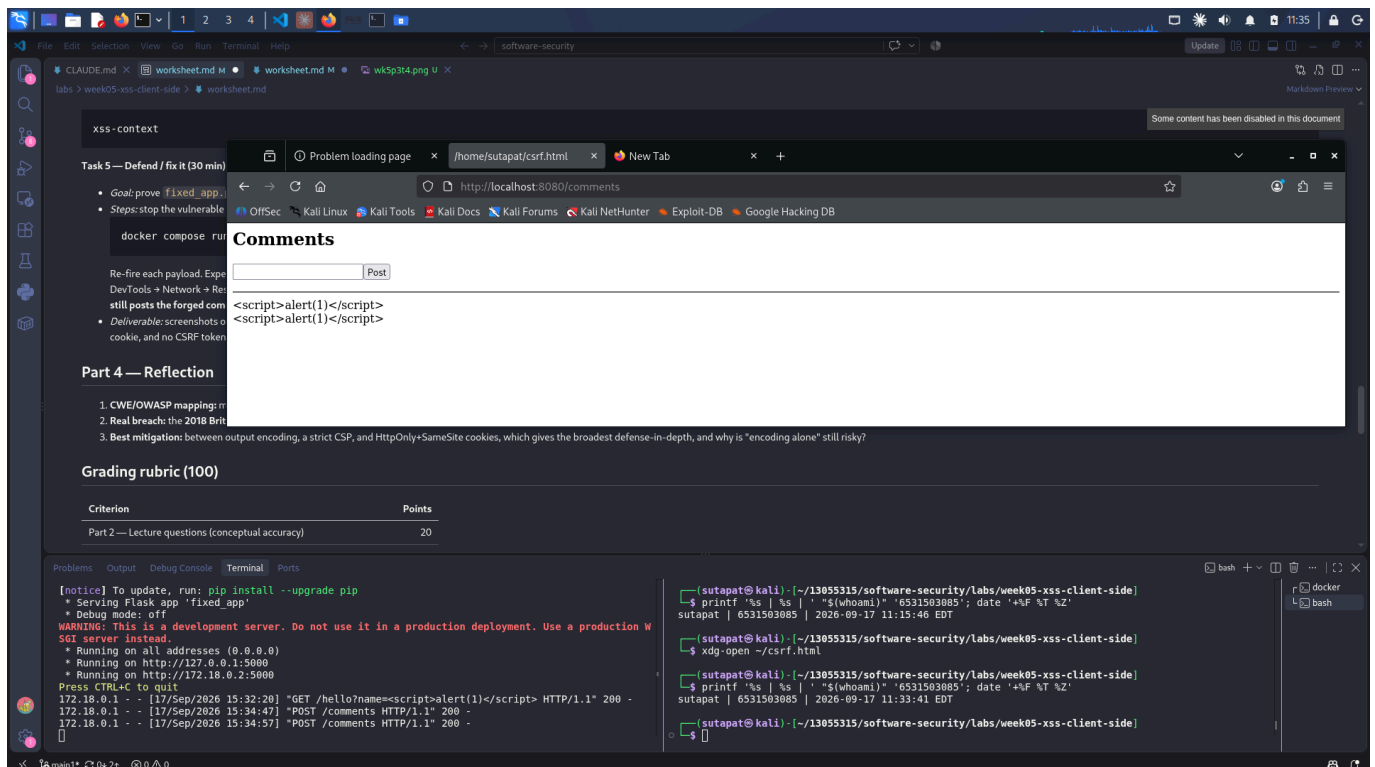
```
Comments



<script>alert(1)</script>
<script>alert(1)</script>
<script>alert(1)</script>
CSRF posted this comment
CSRF posted this comment
```

```
(sutapat@kali) ~/13055315/software-security/labs/week05-xss-client-side
$ printf '%s | %s | ' "$(whoami)" '6531503085'; date +%F %T %Z
sutapat | 6531503085 | 2026-09-17 11:29:11 EDT

(sutapat@kali) ~/13055315/software-security/labs/week05-xss-client-side
$ docker compose run --rm --service-ports xss-lab bash -c "pip install --no-cache-dir flask && python fixed_app.py"
[+] Creating 1/1
✔ Network week05-xss-client-side default Created 0.1s
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker-1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click-8.1.3 (from flask)
  Downloading click-8.1.3 (from flask)
```



```
Comments



<script>alert(1)</script>
<script>alert(1)</script>
```

```
(sutapat@kali) ~/13055315/software-security/labs/week05-xss-client-side
$ printf '%s | %s | ' "$(whoami)" '6531503085'; date +%F %T %Z
sutapat | 6531503085 | 2026-09-17 11:15:46 EDT

(sutapat@kali) ~/13055315/software-security/labs/week05-xss-client-side
$ xdg-open ~/csrf.html

(sutapat@kali) ~/13055315/software-security/labs/week05-xss-client-side
$ printf '%s | %s | ' "$(whoami)" '6531503085'; date +%F %T %Z
sutapat | 6531503085 | 2026-09-17 11:33:41 EDT

(sutapat@kali) ~/13055315/software-security/labs/week05-xss-client-side
$
```

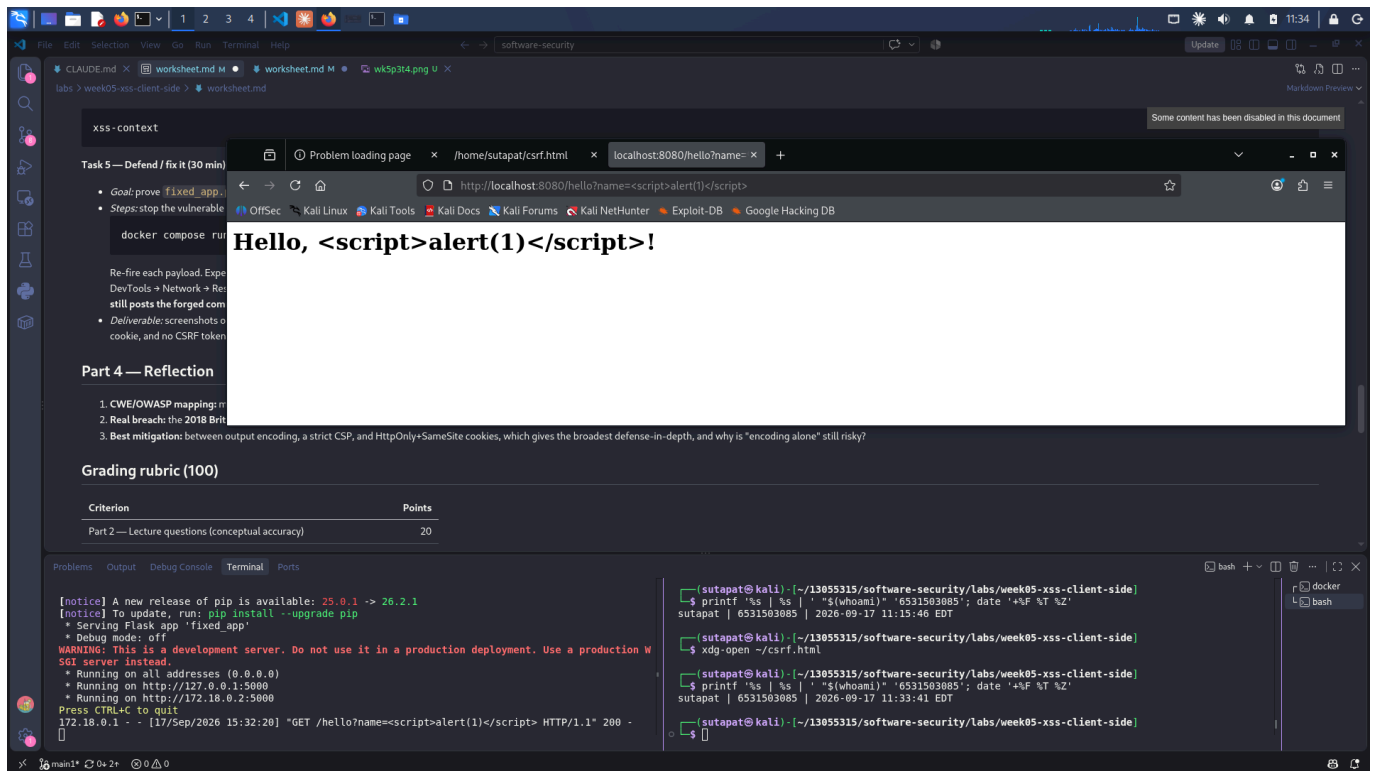
```
[notice] To update, run: pip install --upgrade pip
* Serving Flask app 'fixed_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.2:5000
Press CTRL+C to quit
172.18.0.1 - - [17/Sep/2026 15:32:20] "GET /hello?name=script:alert(1)</script> HTTP/1.1" 200 -
172.18.0.1 - - [17/Sep/2026 15:34:47] "POST /comments HTTP/1.1" 200 -
172.18.0.1 - - [17/Sep/2026 15:34:57] "POST /comments HTTP/1.1" 200 -
```

The screenshot shows a Kali Linux desktop environment. The browser window displays a page with the URL `http://localhost:8080/hello?name=yous`. The DevTools console shows a 200 OK response. The terminal shows the execution of a Docker command to run a Flask application.

```
Container week05-xss-client-side-xss-lab-1 Removed 0.1s
Network week05-xss-client-side default Removed 0.2s
[sutapat@kali:~/13055315/software-security/labs/week05-xss-client-side]
$ print 'xss | %s | ' $(whoami) "6531503085"; date +%F %T %Z
sutapat | 6531503085 | 2026-09-17 11:29:11 EDT
[sutapat@kali:~/13055315/software-security/labs/week05-xss-client-side]
$ docker compose run --rm --service-ports xss-lab bash -c "pip install --no-cache-dir flask && python fixed_app.py"
[+] Creating 1/1
Network week05-xss-client-side_default Created 0.1s
Collecting flask
Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
```

The screenshot shows a Kali Linux desktop environment. The browser window displays a page with the URL `http://localhost:8080/hello?name=yous`. The DevTools console shows a 200 OK response. The terminal shows the execution of a Docker command to run a Flask application.

```
SGI server instead.
* Running on all addresses
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.1:5000
Press CTRL+C to quit
172.18.0.1 - - [17/Sep/2026:15:37:47] "GET /hello?name=yous HTTP/1.1" 200 -
```



Part 4 — Reflection

1. **CWE/OWASP mapping:** map your reflected/stored XSS to **CWE-79** and your CSRF PoC to **CWE-352**, both under OWASP 2025 **A05 Injection** (CSRF historically A01/A05).

Ans: My writings on reflected and stored XSS (Tasks 1–2) both match CWE-79, which is in OWASP 2025's A05 Injection group. Task 4 of my CSRF PoC is related to CWE-352. OWASP 2025 also puts CSRF in A05, but in the past it was more related to A01 Broken Access Control because it uses a missing licence check instead of a standard injection flaw.

2. ****Real breach:**** the ****2018 British Airways breach**** (~380k payment records) used malicious JavaScript (Magecart) injected into the site to skim card data — a client-side script-injection failure. In 3–4 sentences relate it to this lab's XSS and CSP lessons.

Ans: Attackers using Magecart put about 22 lines of malicious JavaScript into British Airways' payment page in 2018 to steal customer card information. This is the same method of silently stealing data as our Task 3 beacon, but they targeted form fields instead of a cookie. This was possible because a script that hadn't been checked could run on the page at all, which is also the main reason why we found what we did in Tasks 1–2. It's possible that the browser would not have been able to send stolen data to an attacker domain that wasn't allowed if the CSP had been strict, like the one we tried in Task 5. About 380,000 credit cards were compromised, and BA was fined £20 million by the ICO.

3. ****Best mitigation:**** between output encoding, a strict CSP, and HttpOnly+SameSite cookies, which gives the broadest defense-in-depth, and why is "encoding alone" still risky?

Ans: A strict CSP is the finest defense-in-depth because it stops an injected script from running or transferring data, even if output encoding is missed. HttpOnly and SameSite secure the cookie, but scripts can modify the page or steal data. If you only encode, one place may go wrong. If you miss an app endpoint, XSS could happen again without a second layer. Encoding to repair the problem at its source, CSP as a safety net, and cookie flags to restrict damage from what goes through are all employed in real life.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	100

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence.** When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (*graded on your reasoning, not copied text*):

1. What did you do, and **why did the vulnerability work?**

To trigger reflected and stored XSS, I injected payloads like and  into unescaped output points (/hello?name= and comments form), stole the session cookie with a beacon script, and forged a cross-site POST to /comments from an external csrf.html page. Because the app trusted user input, it wrote user-supplied content directly into HTML without encoding it and allowed state-changing requests without verifying their source or anti-CSRF token.

2. **Why does your fix actually stop it** — and what could still break it?

Fixed_app.py prevents XSS by autoescaping, turning < and > into innocuous entities. A strict CSP provides a second layer to prohibit any stray scripts. This could still break if a developer bypasses escaping on a new feature (e.g., using |safe) or loosens the CSP with 'unsafe-inline', but as Task 5 showed, /comments never checks a CSRF token or session cookie server-side, therefore the PoC works even against the repaired app.



Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

How do I fix the XSS vulnerability in this Flask route: return f"

Results for: {q}

"?

2. ****Find what's wrong or risky**** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

The AI's patch (Jinja autoescape / markupsafe.escape) is theoretically right for the one /hello reflected-XSS snippet it was shown. However, it is only scoped to that single endpoint. It does not discuss or talk about the DOM-based XSS in /welcome (client-side innerHTML sink, invisible to any server-side remedy) or the CSRF vulnerability on /comments (missing token/auth check, which HTML-escaping cannot cure at all). If you take this answer as "the fix" for the lab, there are still two additional genuine vulnerabilities exploitable.

3. Produce the ****correct, verified**** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

The AI correctly repaired the one excerpt it was given, however XSS-safe output encoding doesn't shut every hole in this experiment, DOM-based XSS resides entirely client-side and CSRF isn't even an encoding concern. A true and widespread blind spot is an AI (or dev) that "fixes the bug you showed it," without verifying that the same class of vulnerability or an entirely different one. exists elsewhere in the app.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.



Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

The name parameter from the URL is sent directly to the HTML answer by the /hello endpoint. It is not encoded or escaped in any way. The browser doesn't know the difference between "text the developer wrote" and "text the user supplied," so when I type in name, it's treated as a real tag and run as JavaScript, not as the words I typed. There is no line between data and code where user input meets HTML output, so it can be used in bad ways.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

In this Flask endpoint, the `name` query parameter is concatenated directly into an HTML response with no escaping, which is vulnerable to reflected XSS (e.g. ?name=):

```
@app.route("/hello") def hello(): name = request.args.get("name", "") return f"
```

Hello, {name}!

"

Give me a corrected version of this route that is safe against XSS for any value of `name`, and explain why it stops both a plain `<script>` tag and an attribute-breakout payload.

Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).